CS 305: THEORY OF COMPUTATION

Week 7: Pushdown Automata & Stack-Based Computation

Sipser Chapter 2.2 · Hierarchy Level 2: Computational Equivalence of CFGs & Stacks

---

Department of Computer Science

School of Technology

Ghana Institute of Management and Public Administration (GIMPA)

Level 300 Undergraduate · Trimester / Semester Schedule

The Bridge from Week 6: How Do We Mechanize Context-Free Grammars?

Week 6 Generative Model: Grammars

Top-Down Syntax Generation

- Context-Free Grammars provide a **generative grammar specification**:

$$S \to 0S1 \mid \varepsilon \implies 0^n1^n$$

- Grammars explain how strings are *produced* via derivations and parse trees.
- But how does an operational machine **decide acceptance** in streaming, linear time?

Week 7 Recognizer Model: Pushdown Automata

Automata Equipped with Memory

- Finite Automata (DFAs/NFAs) failed on 0^n1^n because their memory was strictly **finite** ($|Q|$ states).
- To parse recursive syntax, we attach an **unbounded LIFO stack** to the finite control.
- Result: A **Pushdown Automaton (PDA)**—the exact mechanical dual of a Context-Free Grammar.

The Golden Equation of Context-Free Languages:

Context-Free Language (CFL) $\equiv$ Generated by CFG $\equiv$ Recognized by Pushdown Automaton (PDA)

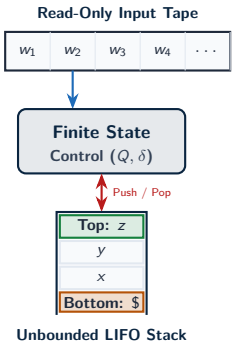
Conceptual Architecture: Finite State Machine + LIFO Stack

The Three Components of a PDA

1. **Input Tape:** Read-only, one-way scan from left to right. Once a symbol is read, the tape head advances.
2. **Finite State Control:** A finite set of internal states Q, switching based on the current state, input symbol, and stack top.
3. **Unbounded LIFO Stack:** A memory store with **Last-In, First-Out** discipline. Only the topmost symbol can be read or popped.

The LIFO Bottleneck

A PDA cannot access symbols buried deep in the stack without popping all elements above them! This makes a stack strictly less powerful than an arbitrary Turing tape.



Formal Definition: The PDA 6-Tuple (Sipser Def 2.13)

Formal Mathematical Definition

A **Pushdown Automaton (PDA)** is formally defined as a 6-tuple:

$$M = \Big(Q,\ \Sigma,\ \Gamma,\ \delta,\ q_0,\ F\Big)$$

1. Q: Finite set of internal **states**.
2. Σ: Finite set called the **input alphabet**.
3. Γ: Finite set called the **stack alphabet** (may include marker symbols like \$).
4. δ: Transition function mapping state, input, and stack top:

$$\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}\Big(Q \times \Gamma_\varepsilon\Big)$$

5. $q_0 \in Q$: Designated **start state**.
6. $F \subseteq Q$: Set of **accept states**.

Notation Alert (Σ_ε and Γ_ε): Defined as $\Sigma \cup \{\varepsilon\}$ and $\Gamma \cup \{\varepsilon\}$. The ε symbol means the machine can transition **without reading input** or **without popping/pushing stack**.

Anatomy of a PDA Transition: The $a, b \to c$ Convention

Transition Arrow Syntax

$$q_i \xrightarrow{a,\, b \to c} q_j$$

Three Mandatory Components

- $a \in \Sigma_\varepsilon$ **(Input):** Symbol read from input tape. If ε, no input symbol is consumed.
- $b \in \Gamma_\varepsilon$ **(Pop):** Symbol required on stack top. It is **popped**. If ε, nothing is popped.
- $c \in \Gamma_\varepsilon$ **(Push):** Symbol written to stack top. It is **pushed**. If ε, nothing is pushed.

Standard Operational Idioms

Operation	Shorthand
Push symbol x on reading a:	$a, \varepsilon \to x$
Pop symbol y on reading a:	$a, y \to \varepsilon$
Inspect y without changing:	$a, y \to y$
Spontaneous push (no input):	$\varepsilon, \varepsilon \to x$
Spontaneous pop (no input):	$\varepsilon, y \to \varepsilon$
Pure DFA move (ignore stack):	$a, \varepsilon \to \varepsilon$

Guard Condition

If transition specifies $b \neq \varepsilon$, the transition **cannot fire** unless symbol b is currently at the top of the stack!

Worked Example 1: Designing a PDA for $L = \{0^n 1^n \mid n \geq 0\}$

Algorithmic Strategy

The Stack as a Reversible Counter

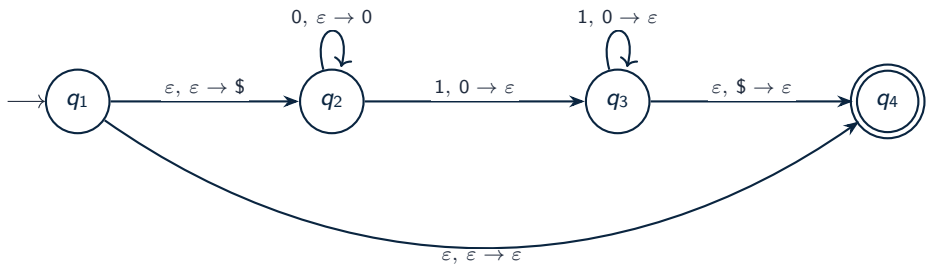
1. **Phase 1 (Initialize):** Push a special marker \$ onto the empty stack so we can detect when the stack is empty later.
2. **Phase 2 (Count 0s):** For every 0 read from input, push a 0 onto the stack.
3. **Phase 3 (Match 1s):** When the first 1 arrives, switch states. For every 1 read, pop a 0 from the stack.
4. **Phase 4 (Verify Acceptance):** When input ends, check if the stack top is \$. If so, pop \$ and transition to accept state!

Why This Defeats the Pumping Lemma

Unbounded Counting via Memory

- A DFA with k states can only count up to k.
- A PDA stack height is **unbounded**: for $0^{10^6}1^{10^6}$, the stack grows to height $10^6 + 1$ and unwinds cleanly.
- If any 0 appears after 1, or if 1s run out before 0s, or if 0s run out before 1s, the machine gets stuck and **rejects**.

State Diagram: PDA for $\{0^n 1^n \mid n \geq 0\}$



Execution Trace on Input $w = 0011$:

$(q_1, 0011, \varepsilon) \vdash (q_2, 0011, \$) \vdash (q_2, 011, 0\$) \vdash (q_2, 11, 00\$) \vdash (q_3, 1, 0\$) \vdash (q_3, \varepsilon, \$) \vdash (q_4, \varepsilon, \varepsilon)$ **[ACCEPTED]**

Formal Semantics: Configurations & The Step Relation ($\vdash$)

Instantaneous Configuration

Configuration Triple (q, w, γ)
A snapshot of a PDA at any computational instant is a triple:

$$(q,\ w,\ \gamma)\ \in\ Q \times \Sigma^* \times \Gamma^*$$

- q: The **current state**.
- w: The **remaining unread input**.
- γ: The **stack contents**, written with the top of stack at the leftmost position!

Single-Step Derivation Relation ($\vdash$)

Step Relation Definition
We write $(q, aw, b\alpha) \vdash (p, w, c\alpha)$ if and only if:

$$(p,\ c)\ \in\ \delta(q,\ a,\ b)$$

where $a \in \Sigma_\varepsilon$, $b, c \in \Gamma_\varepsilon$, and $\alpha \in \Gamma^*$.

- $\vdash^*$: Reflexive transitive closure (multiple computation steps).

Formal Acceptance Condition (Sipser Def 2.15):
A PDA M accepts string $w \in \Sigma^*$ if there exists a sequence of configurations $C_0, C_1, \ldots, C_k$ such that: (1) $C_0 = (q_0, w, \varepsilon)$, (2) $C_i \vdash C_{i+1}$ for all $0 \leq i < k$, and (3) $C_k = (q_f, \varepsilon, \alpha)$ for some $q_f \in F$ and $\alpha \in \Gamma^*$.

Worked Example 2: Balanced Parentheses (Dyck-1 Language)

Problem Formulation

The Language of Nested Brackets

Define $\Sigma = \{(,)\}$. The language of properly matched brackets is:

$$L = \Big\{ w \in \Sigma^* \;\Big|\; \forall \text{ prefix } u, \#_{(}(u) \geq \#_{)}(u), \text{ and } \#_{(}(w) = \#_{)}(w) \Big\}$$

- Valid: ε, (()), (()()), ((()))
- Invalid:)((underflow), (() (unclosed)

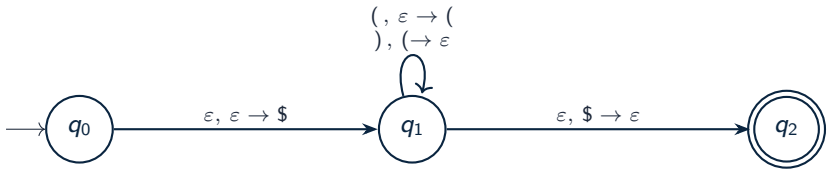
PDA Operational Rules

Stack Discipline

1. Push bottom marker \$ at start.
2. For every open parenthesis '(', **push** '('.
3. For every close parenthesis ')', **pop** '('.
4. If input is empty and stack top is \$, accept.
5. If a ')' arrives when stack top is \$, the machine crashes (**underflow reject**).

Compiler Insight: This exact PDA logic is running inside every IDE, compiler, and JSON parser in the world right now to highlight matching braces and throw syntax errors!

State Diagram: Balanced Parentheses Recognizer



Valid Trace: $w = (())$

$(q_0, (()), \varepsilon) \vdash (q_1, (()), \$) \vdash (q_1, ()), (\$) \vdash (q_1,)), ((\$)$
$\vdash (q_1,), (\$) \vdash (q_1, \varepsilon, \$) \vdash (q_2, \varepsilon, \varepsilon)$ **[ACCEPT]**

Invalid Trace: $w =)($

$(q_0,)(, \varepsilon) \vdash (q_1,)(, \$) \rightarrow$ Next input is ')', but stack top is \$ (requires '('). No transition exists $\implies$ **HALT & REJECT!**

Nondeterminism in PDAs: Why DPDA $\subsetneq$ NPDA

Finite Automata Contrast

Regular World: No Extra Power

- In Level 3 (Regular Languages):

$$\text{DFA} \equiv \text{NFA}$$

- Every NFA with k states can be converted into an equivalent DFA with $\leq 2^k$ states via subset construction.
- Nondeterminism offers convenience, **not power**.

Pushdown Automata Reality

Context-Free World: Strictly More Power!

- In Level 2 (Context-Free Languages):

$$\textbf{DPDA} \subsetneq \textbf{NPDA}$$

- Deterministic PDAs (DPDAs) recognize **Deterministic CFLs (DCFLs)**.
- NPDAs recognize **All Context-Free Languages (CFLs)**.
- There exist CFLs that **no deterministic PDA can parse**!

Why Does Subset Construction Fail for PDAs? In an NFA, you track a subset of states ($2^{|Q|}$). But an NPDA would have to track a subset of **different unbounded stacks**! You cannot merge multiple distinct stacks into a single deterministic stack.

Worked Example 3: Even-Length Palindromes $L = \{ww^R \mid w \in \{0,1\}^*\}$

The Midpoint Dilemma

Why a DPDA Fails on ww^R

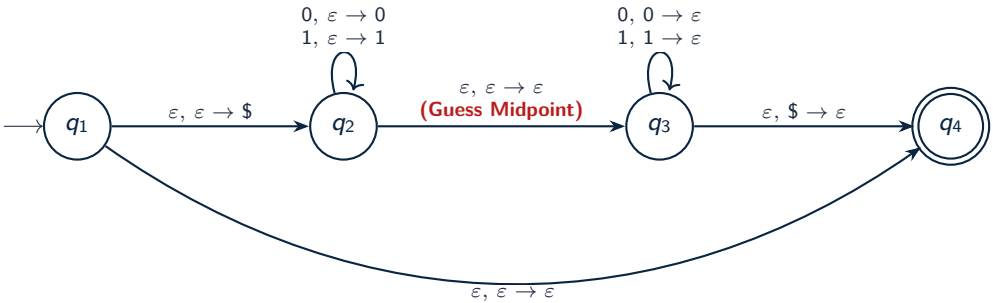
- Consider input 000000. Is the midpoint at index 1, 2, or 3?
- For marked palindromes $w\#w^R$, the delimiter # tells a DPDA exactly when to stop pushing and start popping.
- But for unmarked ww^R, **there is no delimiter**! A deterministic machine cannot know when the first half ends.

The NPDA Solution: Guessing the Midpoint

Spontaneous ε-Transition

- **Push Phase (q_{push}):** Read input and push onto stack.
- **The Guess ($\varepsilon, \varepsilon \to \varepsilon$):** At *every single character*, non-deterministically spawn a branch that guesses: *"We have reached the midpoint!"*
- **Pop Phase (q_{pop}):** Pop and match remaining input against stack top.
- The correct branch survives; all incorrect guesses halt and die!

State Diagram: NPDA for Unmarked Palindromes $\{ww^R\}$



Parallel Branching Tree on String $w = 0110$:

- **Branch 1 (Guess midpoint after 0):** Pops on 1, but stack top is 0 $\implies$ **Dies**.
- **Branch 2 (Guess midpoint after 01):** Stack has 10\$. Matches next 1 (pops 1), matches next 0 (pops 0), pops \$ $\implies$ **ACCEPTS!**
- **Branch 3 (Never guesses midpoint):** Pushes all symbols until tape ends $\implies$ **Dies**.

The Fundamental Equivalence Theorem (Sipser Theorem 2.20)

Chomsky–Schützenberger Equivalence Theorem (1963)

A language L is context-free $\iff$ Some pushdown automaton recognizes L

Direction 1: CFG $\implies$ PDA ($\Leftarrow$)

Top-Down Parser Simulator

- Given any CFG G, we construct an NPDA P that simulates leftmost derivations on its stack.
- The stack holds the unexpanded non-terminals and terminals.
- *Proof is constructive and universal!*

Direction 2: PDA $\implies$ CFG ($\Rightarrow$)

Bottom-Up State Pair Grammars

- Given any PDA P, construct a grammar where variables A_{pq} represent the computation that takes P from state p with empty stack to state q with empty stack.
- Proves that stack memory cannot compute anything outside CFLs.

Universal Construction: Converting Any CFG into a PDA

The 3-Step Universal Top-Down Parser

Given CFG $G = (V, \Sigma, R, S)$, construct a PDA with alphabet $\Gamma = V \cup \Sigma \cup \{\$\}$:

1. **Initialize:** Push \$ and then push start variable S.
2. **Loop State (q_{loop}):**
   - **Rule Expansion:** If stack top is variable $A \in V$, non-deterministically replace with RHS of rule $A \to u$:

     $$\varepsilon,\ A\ \to\ u^R \quad \text{(push RHS in reverse so first symbol is on top!)}$$

   - **Terminal Match:** If stack top is terminal $a \in \Sigma$, read a from input and pop:

     $$a,\ a\ \to\ \varepsilon$$

3. **Accept:** If stack top is \$, pop \$ and enter accept state: $\varepsilon,\ \$ \to \varepsilon$.

Parsing Insight: This construction proves that **parsing is stack simulation**. The PDA stack stores the pending frontiers of the leftmost derivation!

Worked Example 4: CFG to PDA in Action

Input Grammar G

$$S \rightarrow 0S1 \mid \varepsilon$$

Alphabet: $\Sigma = \{0, 1\}$
Variables: $V = \{S\}$

Resulting PDA Transitions

- Init: $\varepsilon, \varepsilon \rightarrow S\$$
- Expansions: $\varepsilon, S \rightarrow 0S1 \mid \varepsilon$
- Terminal matches: $0, 0 \rightarrow \varepsilon$ and $1, 1 \rightarrow \varepsilon$
- Finish: $\varepsilon, \$ \rightarrow \varepsilon$

Trace on Input $w = 01$

State	Input	Stack	Transition Applied
q_{start}	01	ε	$\varepsilon, \varepsilon \rightarrow \$$
q_{loop}	01	\$	$\varepsilon, \varepsilon \rightarrow S$
q_{loop}	01	$S\$$	$\varepsilon, S \rightarrow 0S1$ (Expand S)
q_{loop}	01	$0S1\$$	$0, 0 \rightarrow \varepsilon$ (**Match 0**)
q_{loop}	1	$S1\$$	$\varepsilon, S \rightarrow \varepsilon$ (Expand $S \rightarrow \varepsilon$)
q_{loop}	1	$1\$$	$1, 1 \rightarrow \varepsilon$ (**Match 1**)
q_{loop}	ε	\$	$\varepsilon, \$ \rightarrow \varepsilon$ (**Success**)
q_{accept}	ε	ε	**ACCEPTED!**

Deterministic Context-Free Languages (DCFLs) & Compilers

Definition of a DPDA

Deterministic Constraint (Sipser 2.4)
A PDA M is **deterministic (DPDA)** if:

1. For every $q \in Q, a \in \Sigma_\varepsilon, b \in \Gamma_\varepsilon$, $|\delta(q, a, b)| \leq 1$.
2. If $\delta(q, a, b) \neq \emptyset$, then $\delta(q, \varepsilon, b) = \emptyset$ (no ambiguity between reading and ε-moves).

Why Industry Demands DCFLs

Linear-Time Compilation $\mathcal{O}(n)$

- General CFLs require $\mathcal{O}(n^3)$ parsing (CYK / Earley algorithms).
- **DCFLs parse in guaranteed $\mathcal{O}(n)$ time** with zero backtracking!
- Modern programming languages (C, Rust, Python, Go) are strictly engineered to be **DCFLs** parsed by LR(k) / LALR parsers (e.g., Yacc, Bison).

The Strict Hierarchy: REG $\subsetneq$ DCFL $\subsetneq$ CFL. Example: $\{a^n b^n\} \in$ DCFL, but palindromes $\{ww^R\} \in$ CFL $\setminus$ DCFL.

Acceptance Criteria: Final State vs. Empty Stack

Acceptance by Final State $L(M)$

Standard Sipser Model

$$L(M) = \{w \mid (q_0, w, \varepsilon) \vdash^* (q_f, \varepsilon, \alpha),\ q_f \in F\}$$

- Reaches accept state $q_f \in F$ after consuming all input.
- Stack **does not need to be empty** ($\alpha \in \Gamma^*$).

Acceptance by Empty Stack $N(M)$

Null Stack Model

$$N(M) = \{w \mid (q_0, w, \varepsilon) \vdash^* (q, \varepsilon, \varepsilon),\ q \in Q\}$$

- Consumes all input and simultaneously **clears stack completely**.
- Accept states are not required.

Equivalence Theorem: For nondeterministic PDAs, acceptance by final state and by empty stack are computationally equivalent:

$$\mathcal{L}_{\text{final}}(\text{NPDA}) \equiv \mathcal{L}_{\text{empty}}(\text{NPDA}) \equiv \text{CFL}$$

Closure Properties: Where CFLs Break Down

CFLs ARE Closed Under:

Positive Closure Operations

- **Union ($L_1 \cup L_2$):** Non-deterministically branch to start of M_1 or M_2.
- **Concatenation ($L_1 L_2$):** When M_1 finishes, pop stack and transition to M_2.
- **Kleene Star (L^*):** Loop accept states of M back to start state.

CFLs ARE NOT Closed Under:

Catastrophic Non-Closure!

- **Intersection ($L_1 \cap L_2$): NOT CLOSED!**

$$L_1 = \{a^n b^n c^m\},\ L_2 = \{a^m b^n c^n\} \in \text{CFL}$$

$$L_1 \cap L_2 = \{a^n b^n c^n\} \notin \textbf{CFL!}$$

- **Complement ($\overline{L}$): NOT CLOSED!** (By De Morgan: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$).

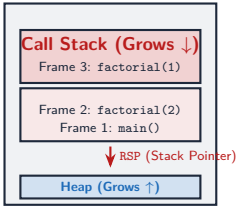
The One-Stack Limitation: A PDA has exactly **one stack**. It can count as vs bs, or bs vs cs, but once that stack is unwound, it cannot count a third independent power! Tracking two independent stacks equals a **Turing Machine**.

Modern Spotlight: Runtime Call Stacks in Hardware & OS

The Physical PDA in Your CPU

- When an x86/ARM CPU executes programs, it allocates a dedicated **Runtime Call Stack** managed by the **Stack Pointer ('RSP'/'SP')**.
- **Function Invocation $\implies$ PUSH:**
  - Push return address to stack.
  - Allocate stack frame for local variables.
- **Function Return $\implies$ POP:**
  - Deallocate frame.
  - Pop return address into Instruction Pointer.

RAM: Process Address Space



Recursive Function Proof: Why can't a language without a call stack support recursion? Because without LIFO storage of return addresses, a function calling itself overwrites its own activation records!

Systems Security: Buffer Overflows & Smashing the Stack

Vulnerability: Unchecked LIFO Writes

Smashing the Stack for Fun & Profit

- In C/C++, local arrays sit on the stack right next to the saved **frame pointer** and **return address**.
- Overflowing buffer via unsafe ‘gets()‘:

  $$\text{Buffer} \rightarrow \text{EBP} \rightarrow \textbf{Return Addr}$$

- Overwrites return address $\Longrightarrow$ CPU jumps to **shellcode**!

Modern Architectural Defenses

Enforcing Automata Integrity

1. **Stack Canaries (‘-fstack-protector‘):** Random secret integer before return address; OS aborts if altered.
2. **Non-Executable Stack (NX / W^X):** Marks stack pages non-executable, stopping shellcode.
3. **Shadow Stacks (Intel CET):** Hardware secondary stack storing read-only return address copies.

Interactive Checkpoint: PDA Design Challenge

Class Challenge: Multi-Option Counting

Problem: Consider the language $L = \left\{ a^i b^j c^k \;\middle|\; i, j, k \geq 0 \text{ and } (i = j \text{ or } i = k) \right\}$.

1. Is L recognizable by a **Deterministic** Pushdown Automaton (DPDA)?
2. How does a **Nondeterministic** Pushdown Automaton (NPDA) handle this language?

Solution Walkthrough:

- **Answer 1: NO.** When reading the a's, a machine cannot predict whether to match them against the b's or skip the b's and match against the c's. Hence, $L \notin$ DCFL.
- **Answer 2: NPDA branches spontaneously at the start!**
  - *Branch A:* Push a's on stack, pop and match against b's, then ignore all c's.
  - *Branch B:* Push a's on stack, ignore all b's without touching stack, then pop and match against c's!
- If either condition holds, that branch reaches accept. This exemplifies the power of **nondeterministic guess branching**.

Common Exam Traps & Pitfalls to Avoid

Trap 1: Confusing Σ and Γ

The input alphabet Σ and stack alphabet Γ are **distinct sets**. You can push symbols onto the stack that never appear in the input string (e.g., markers like \$, counters like X).

Trap 2: Forgetting the Bottom Marker

A stack does not automatically know when it is empty. If you do not push a bottom marker like \$ at the start, your machine cannot safely test for an empty stack!

Trap 3: Assuming DPDA $\equiv$ NPDA

Never write that a DPDA can recognize any CFL. While DFA $\equiv$ NFA, nondeterminism gives PDAs **strictly greater power** (DCFL $\subsetneq$ CFL).

Trap 4: Claiming CFLs Are Closed Under $\cap$

If an exam question asks whether $\{a^n b^n c^n\}$ is context-free, **it is NOT**. Intersecting two CFLs often requires two stacks, which is a Turing Machine!

Roadmap to Midterm Examination (Week 8)

Midterm Exam Scope (Modules 1 & 2)

Key Exam Competencies

- **Regular Languages (Weeks 1–5):** DFA/NFA formal 5-tuples, subset construction, RegEx $\longleftrightarrow$ GNFA conversions, and Regular Pumping Lemma proofs.
- **Context-Free Grammars (Week 6):** Grammar design, leftmost derivations, parse trees, ambiguity proofs, and 4-step Chomsky Normal Form (CNF) conversion.
- **Pushdown Automata (Week 7):** Formal 6-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$, transition conventions $(a, b \to c)$, CFG-to-PDA equivalence, and DPDA vs NPDA power.

Week 8 Lecture Preview

Non-Context-Free Languages

- **The Pumping Lemma for CFLs:**

$$s = uvxyz \quad (|vxy| \leq p,\ |vy| \geq 1)$$

- Splitting strings into 5 pieces to prove languages non-context-free.
- Showing $\{a^n b^n c^n\}$ and $\{ww\}$ cannot be recognized by any PDA.
- **Modern Spotlight:** Why transformer attention struggles with deeply nested Dyck-k structures.

Week 7 Synthesis & Milestone Checklist

Summary of Key Takeaways

- **Pushdown Automaton:** Finite state control combined with an unbounded LIFO stack: $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$.
- **Transition Shorthand:** $a, b \to c$ means: read a from input, pop b from stack, push c onto stack.
- **Chomsky Equivalence:** Every Context-Free Grammar has an equivalent Pushdown Automaton ($L(G) = L(M)$).
- **Determinism Divide:** Unlike finite automata, DPDA $\subsetneq$ NPDA. Programming languages are restricted to DCFLs to guarantee $\mathcal{O}(n)$ parsing.
- **Closure Limits:** CFLs are closed under $\cup, \circ, *$, but **NOT** closed under $\cap$ or complementation.

Action Item: Review Modules 1 and 2 in Sipser (Chapters 1 & 2) for the upcoming Midterm Examination.

Milestone: Problem Set 2 solutions released. Review Pumping Lemma & CFG ambiguity proofs.